

基于遗传算法(GA)的公共交通工具调度优化研究

摘要

随着城市公共交通系统的快速发展,公交车辆调度优化成为提升城市交通效率和服务质量的关键问题。合理的发车间隔安排能够有效减少乘客等待时间,降低车站拥挤程度,提高公交系统的整体运行效率。然而,公交调度问题受到多种实际约束条件的限制,包括最小/最大发车间隔、车辆容量限制、车站客流容量以及随时间变化的乘客需求等,使得这一问题变得异常复杂。

针对复杂约束条件下的公交车辆发车间隔优化问题,本文提出了一种基于遗传算法(Genetic Algorithm, GA)的优化方法。通过染色体编码表示发车时间,设计适应度函数综合考虑乘客等待时间和车站延误人数,利用选择、交叉和变异操作进化种群,最终得到最优的发车时间表。实验结果表明,该方法能够有效降低乘客总等待时间,显著改善公交系统的服务质量。

关键词: 公交调度; 遗传算法; 发车间隔优化; 等待时间最小化

Abstract

With the rapid development of urban public transportation systems, bus scheduling optimization has become a key issue in improving urban traffic efficiency and service quality. Reasonable departure intervals can effectively reduce passenger waiting times, decrease station congestion, and enhance the overall operational efficiency of bus systems. However, the bus scheduling problem is subject to various practical constraints, including minimum/maximum departure intervals, vehicle capacity limits, station passenger capacity, and time-varying passenger demand, making this problem exceptionally complex.

To address the bus departure interval optimization problem under complex constraints, this paper proposes a genetic algorithm (GA)-based optimization method. By using chromosome encoding to represent departure times, designing a fitness function that comprehensively considers passenger waiting times and station delay numbers, and utilizing selection, crossover, and mutation operations to evolve the population, an optimal departure schedule is ultimately obtained. Experimental results show that this method can effectively reduce the total passenger waiting time and significantly improve the service quality of the bus system.

Keywords: Bus scheduling; Genetic algorithm; Departure interval optimization; Waiting time minimization

目录

1.引言.....1

 1.1 研究背景.....1

 1.2 研究意义.....1

2.问题描述与建模.....1

 2.1 问题描述.....1

 2.2 数学模型.....2

3. 数据集描述.....3

 3.1 客流数据结构.....3

 3.2 客流特征分析.....3

4. 遗传算法设计与实现.....4

 4.1 算法框架.....4

 4.2 算法参数设置.....4

 4.3 核心模块实现.....4

 4.4 主程序流程.....8

5. 实验结果与分析.....10

 5.1 算法收敛性分析.....10

 5.2 关键站点分析.....11

 5.4 优化效果分析.....12

7. 讨论.....12

 7.1 算法优势.....12

 7.2 存在的问题与改进方向.....13

 7.3 实际应用考虑.....13

8. 结论.....13

1.引言

1.1 研究背景

随着城市化进程的加快和人口密度的增加，公共交通系统在城市交通体系中扮演着越来越重要的角色。高效的公共交通调度不仅能够提高运输效率，减少乘客等待时间，还能缓解城市交通拥堵，降低环境污染。然而，公共交通车辆调度是一个复杂的组合优化问题，涉及多个约束条件和优化目标，传统的数学规划方法往往难以在合理时间内获得满意解。

城市公共交通系统的车辆调度问题具有以下特点：

多约束性：需要考虑车辆容量限制、发车间隔约束、站点通行能力等多种约束条件

动态性：客流量随时间变化，早晚高峰期客流分布差异显著

复杂性：属于 NP-hard 问题，随着站点数量和车辆数量的增加，问题复杂度呈指数级增长

实时性：需要快速生成可行的调度方案，以应对实际运营中的各种情况

1.2 研究意义

本研究的意义主要体现在以下几个方面：

理论意义：为公共交通调度问题提供了一种基于进化计算的求解框架，丰富了智能优化算法在交通领域的应用研究。

实践意义：所提出的方法可以直接应用于实际的公共交通调度系统，帮助交通管理部门制定更加合理的发车计划，提高服务质量。

经济效益：通过优化调度方案，可以在不增加车辆投入的情况下，提高运输效率，降低运营成本。

社会效益：减少乘客等待时间，改善出行体验，提高公共交通的吸引力，促进绿色出行。

2.问题描述与建模

2.1 问题描述

本研究考虑的是一个典型的城市公共交通线路调度问题。该线路包含 20 个站点，需要在早高峰时段（6:30-8:30）安排 25 辆公交车的发车时间。问题的核

心是在满足各种运营约束的条件下，制定一个发车时间表，使得整个系统的服务质量最优。

具体问题参数如下：

站点数量：20 个站点（StationNum=20）

车辆数量：25 辆公交车（carNum=25）

运营时间窗口：6:30-8:30（Ts=390 分钟，Te=510 分钟）

发车间隔约束：最小间隔 2 分钟（minT=2），最大间隔 8 分钟（maxT=8）

车辆容量：每辆车最大载客量 1860 人（MaxCapacity=1860）

限流站点：第 5 站和第 13 站实施客流限制，每站限制 600 人

2.2 数学模型

2.2.1 决策变量

设决策变量为

$$X = [x_1, x_2, \dots, x_{25}]$$

其中 x_i 表示第 i 辆车的发车时间，满足：

$$x_1 = Ts = 390 \text{（第一辆车在 6:30 发车）}$$

$$x_{25} = Te = 510 \text{（最后一辆车在 8:30 发车）}$$

$$x_{i+1} - x_i \in [2, 8], \forall i \in \{1, 2, \dots, 24\}$$

2.2.2 目标函数

目标函数为最小化总的服务质量损失，包括乘客等待时间和站台滞留人数：

$$\min f(X) = \sum_{s=1}^{20} \sum_{c=1}^{25} (W_{s,c} + D_{s,c})$$

其中：

$W_{s,c}$ 表示第 s 个站点第 c 辆车发车后的等待人数

$D_{s,c}$ 表示第 s 个站点第 c 辆车到达时因限流未能上车的人数

2.2.3 约束条件

发车间隔约束：

$$2 \leq x_{i+1} - x_i \leq 8, \forall i \in \{1, 2, \dots, 24\}$$

车辆容量约束：

$$L_c \leq 1860, \forall c \in \{1, 2, \dots, 25\}$$

站点限流约束：对于限流站点 $s \in \{5, 13\}$ ，每次上车人数不超过 600 人

3. 数据集描述

3.1 客流数据结构

研究使用的数据集包含各站点在不同时段的客流量信息，存储在"数据 1.xlsx"文件中。数据结构如下：

第 1 列：站点编号（1-20）

第 2-4 列：不同时间段的每分钟到站客流量

第 2 列：6:30-7:00 时段客流量

第 3 列：7:00-8:00 时段客流量

第 4 列：8:00-8:30 时段客流量

最后一列：各站点的目标下车人数分布

	A	B	C	D	E	
1	车站到达	6:30-7:00	7:00-8:00	8-8:30	车站下车	
2	1	40	50	40		
3	2	30	40	30	1	
4	3	40	50	40	2	
5	4	40	50	40	2	
6	5	50	60	50	3	
7	6	30	40	30	1	
8	7	40	50	40	2	
9	8	40	50	40	2	
10	9	40	50	40	3	
11	10	30	40	30	1	
12	11	30	40	30	1	
13	12	30	40	30	1	
14	13	50	60	50	2	
15	14	40	50	40	2	
16	15	40	50	40	2	
17	16	40	50	40	2	
18	17	40	50	40	2	
19	18	30	40	30	1	
20	19	30	40	30	1	
21	20				1	

3.2 客流特征分析

根据代码中的数据处理逻辑，客流呈现以下特征：

时变特性：客流量随时间变化，早高峰期间客流量逐渐增加

空间分布：不同站点的客流量存在差异，部分站点（如第 5 站和第 13 站）客流压力较大

下车规律：采用递减式下车率模型，即：

$$getOffRate_s = \frac{targetDistribute_s}{\sum_{i=s}^{20} targetDistribute_i}$$

4. 遗传算法设计与实现

4.1 算法框架

遗传算法是一种基于自然选择和遗传机制的启发式搜索算法，特别适合求解复杂的组合优化问题。本研究设计的遗传算法框架包括以下核心组件：

染色体编码：采用实数编码，每个染色体表示一个完整的发车时间方案

种群初始化：生成满足约束条件的初始种群

适应度评价：基于目标函数计算个体适应度

遗传操作：包括交叉和变异操作

修复机制：确保生成的解满足所有约束条件

选择策略：采用精英保留策略

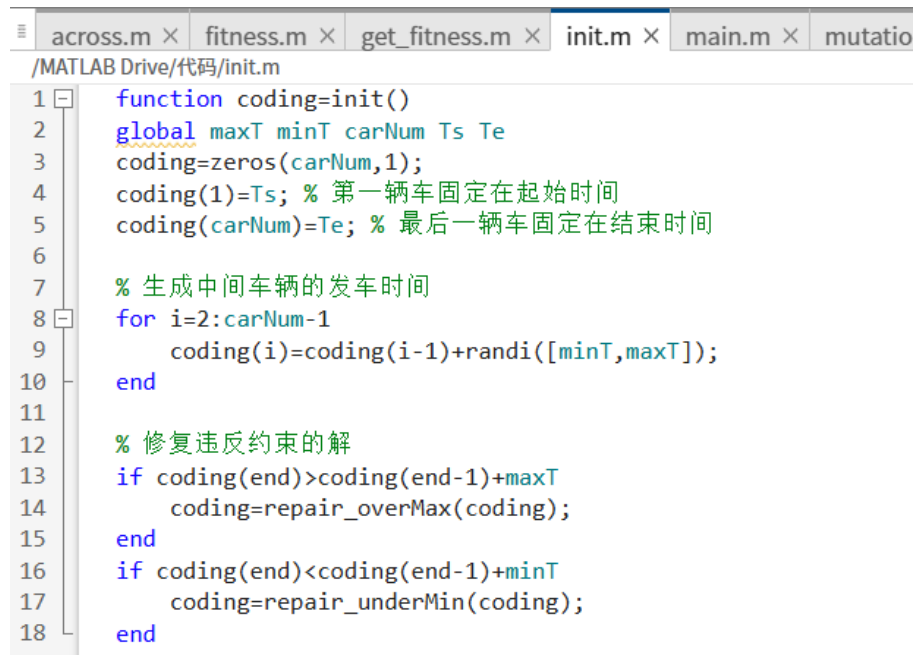
4.2 算法参数设置

```
120 %% GA参数设置
121 Maxgen = 100; % 最大迭代次数
122 Popnum = 20; % 种群规模
123 Pc = 0.5; % 交叉概率
124 Pm = 0.5; % 变异概率
125 % ObjFunc=@(x)fitness(x);
```

4.3 核心模块实现

4.3.1 种群初始化模块（init.m）

初始化模块负责生成满足约束条件的初始解，首先固定首末车辆的发车时间，通过随机生成[2,8]之间的间隔来确定中间车辆的发车时间，最后通过修复函数确保所有间隔满足约束：



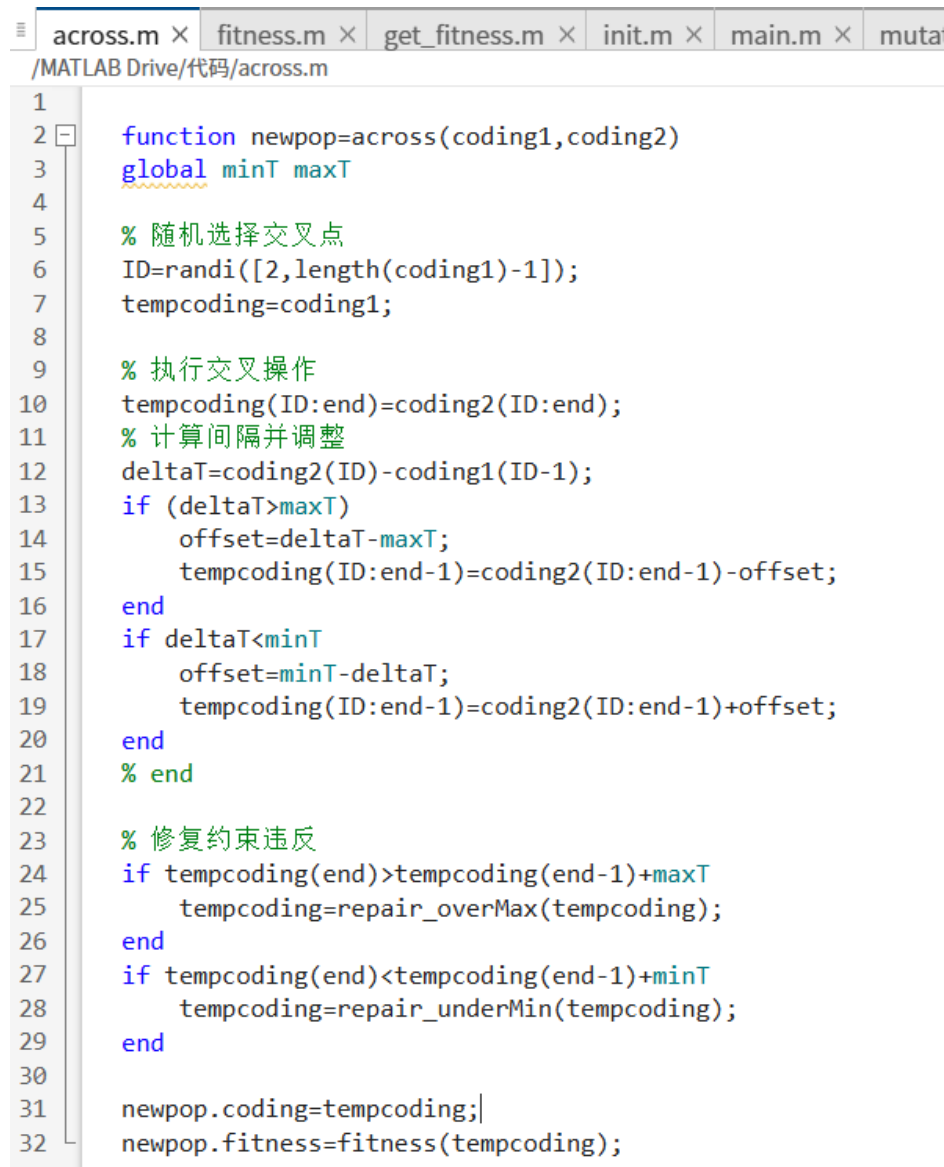
```

1 function coding=init()
2 global maxT minT carNum Ts Te
3 coding=zeros(carNum,1);
4 coding(1)=Ts; % 第一辆车固定在起始时间
5 coding(carNum)=Te; % 最后一辆车固定在结束时间
6
7 % 生成中间车辆的发车时间
8 for i=2:carNum-1
9     coding(i)=coding(i-1)+randi([minT,maxT]);
10 end
11
12 % 修复违反约束的解
13 if coding(end)>coding(end-1)+maxT
14     coding=repair_overMax(coding);
15 end
16 if coding(end)<coding(end-1)+minT
17     coding=repair_underMin(coding);
18 end

```

4.3.2 交叉操作模块 (across.m)

交叉操作采用单点交叉策略, 在保持约束有效性的同时交换两个父代的基因片段, 随机选择一个交叉点 (第 2 到第 24 个位置), 将第二个父代从交叉点之后的基因复制到子代, 检查交叉点处的时间间隔是否满足约束, 如不满足则进行平移调整, 最后对整个染色体进行约束修复:



```

1
2 function newpop=across(coding1,coding2)
3 global minT maxT
4
5 % 随机选择交叉点
6 ID=randi([2,length(coding1)-1]);
7 tempcoding=coding1;
8
9 % 执行交叉操作
10 tempcoding(ID:end)=coding2(ID:end);
11 % 计算间隔并调整
12 deltaT=coding2(ID)-coding1(ID-1);
13 if (deltaT>maxT)
14     offset=deltaT-maxT;
15     tempcoding(ID:end-1)=coding2(ID:end-1)-offset;
16 end
17 if deltaT<minT
18     offset=minT-deltaT;
19     tempcoding(ID:end-1)=coding2(ID:end-1)+offset;
20 end
21 % end
22
23 % 修复约束违反
24 if tempcoding(end)>tempcoding(end-1)+maxT
25     tempcoding=repair_overMax(tempcoding);
26 end
27 if tempcoding(end)<tempcoding(end-1)+minT
28     tempcoding=repair_underMin(tempcoding);
29 end
30
31 newpop.coding=tempcoding;
32 newpop.fitness=fitness(tempcoding);

```

4.3.3 变异操作模块（mutation.m）

变异操作通过微调某个发车时间来增加种群多样性，随机选择一个变异点，以 50%的概率将该点及其后续所有发车时间延后或提前 1 分钟，变异操作保证不违反间隔约束，变异后进行约束检查和修复：


```

across.m × fitness.m × get_fitness.m × init.m × main.m × mutation.
/MATLAB Drive/代码/mutation.m
1 function newpop=mutation(coding)
2 global minT maxT
3
4 % 随机选择变异位置
5 ID=randi([2,length(coding)-1]);
6 deltaT=coding(ID)-coding(ID-1);
7
8 % 随机增加或减少1分钟
9 if rand()<0.5
10     if deltaT<=maxT-1
11         coding(ID:end-1)=coding(ID:end-1)+1; % 延后1分钟
12     end
13 else
14     if deltaT>=minT+1
15         coding(ID:end-1)=coding(ID:end-1)-1; % 提前1分钟
16     end
17 end
18
19 % 修复约束违反
20 if coding(end)>coding(end-1)+maxT
21     coding=repair_overMax(coding);
22 end
23 if coding(end)<coding(end-1)+minT
24     coding=repair_underMin(coding);
25 end
26
27 newpop.coding=coding;
28 newpop.fitness=fitness(coding);
29

```

4.3.4 约束修复模块

修复模块确保生成的解始终满足所有约束条件：

repair_overMax.m（修复最大间隔违反）：

```

across.m × fitness.m × get_fitness.m × init.m × main.m × muta
/MATLAB Drive/代码/repair_overMax.m
1 function coding=repair_overMax(coding)
2 global maxT
3
4 while 1
5     ID=randi([2,length(coding)-1]);
6     deltaT=coding(ID)-coding(ID-1);
7     if deltaT<maxT-1
8         coding(ID:end-1)=coding(ID:end-1)+1; % 整体后移
9     end
10    if coding(end)<=coding(end-1)+maxT
11        break % 约束满足，退出循环
12    end
13 end

```

repair_underMin.m (修复最小间隔违反):

```

across.m × fitness.m × get_fitness.m × init.m × main.m × mutat
/MATLAB Drive/代码/repair_underMin.m
1 function coding=repair_underMin(coding)
2   global maxT minT
3
4   while 1
5       ID=randi([2,length(coding)-1]);
6       deltaT=coding(ID)-coding(ID-1);
7       if deltaT<maxT-1
8           coding(ID:end-1)=coding(ID:end-1)+1; % 整体后移
9       end
10      if coding(end)<coding(end-1)+minT
11          break % 约束满足，退出循环
12      end
13  end

```

4.4 主程序流程

主程序协调各个模块，实现完整的遗传算法流程:

```

across.m × fitness.m × get_fitness.m × init.m × main.m × mutation.m × plotresult.m × repair_overMax.m ×
/MATLAB Drive/代码/main.m
1 clc
2 close all
3 clear all
4
5 global StationNum maxT minT carNum Ts Te carNum Tlist passengerNumMinit getOffRate MaxCapacity
6
7 % StationNum=20;
8 % 参数初始化
9 maxT=8;
10 minT=2;
11 StationNum=20;
12 carNum=25;
13 MaxCapacity=1860;
14 Ts=6.5*60;
15 Te=8.5*60;
16 Tstep=1;
17 Tlist=[Ts:Tstep:Te];
18
19 % 数据读取与预处理
20 path = '数据1.xlsx';
21
22 % 检查文件是否存在
23 if ~exist(path, 'file')
24     error('错误: 文件 "%s" 不存在。请确保文件在MATLAB当前工作目录中。', path);
25 end
26
27 try
28     % 读取Excel文件
29     dataTable = readtable(path);
30
31     % 提取数值数据 (跳过第一行标题)
32     data = table2array(dataTable(2:end, 2:end)); % 从第二行开始，第二列到最后一列
33
34     % 处理NaN值
35     data(isnan(data)) = 0;
36 catch ME
37     error('读取Excel文件时出错: %s', ME.message);
38 end
39

```

```

40    %% 乘客流量分布
41    passengerNum = data(:, 1:3); % 前三列是三个时间段的乘客数据
42    passengerNumMinit = zeros(StationNum, length(Tlist));
43
44    for i = 1:length(Tlist)
45        if Tlist(i) <= 7*60 % 6:30-7:00
46            passengerNumMinit(:, i) = passengerNum(:, 1);
47        elseif Tlist(i) <= 8*60 % 7:00-8:00
48            passengerNumMinit(:, i) = passengerNum(:, 2);
49        else % 8:00-8:30
50            passengerNumMinit(:, i) = passengerNum(:, 3);
51        end
52    end
53    passengerNumMinit = passengerNumMinit * 1.1; % 增加10%的乘客量
54
55    %% 到站分布
56    % 提取下车比例数据（最后一列）
57    targetDistribute = data(:, end);
58    getOffRate = zeros(size(targetDistribute));
59
60    for i = 1:StationNum
61        getOffRate(i) = targetDistribute(i) / sum(targetDistribute(i:end));
62    end %% 人流量分布
63    passengerNum=data(:,2:4);
64    passengerNumMinit=zeros(StationNum,length(Tlist));
65    for i=1:length(Tlist)
66        if Tlist(i)<=7*60
67            passengerNumMinit(:,i)=passengerNum(:,1);
68        elseif Tlist(i)<=8*60
69            passengerNumMinit(:,i)=passengerNum(:,2);
70        else
71            passengerNumMinit(:,i)=passengerNum(:,3);
72        end
73    end
74    passengerNumMinit=passengerNumMinit*1.1;
75
76    %% 到站分布
77    targetDistribute=data(:,end);
78    getOffRate=zeros(size(targetDistribute));
79    for i=1:StationNum
80        getOffRate(i)=targetDistribute(i)/sum(targetDistribute(i:end));
81    end

```

```

120 %% GA参数设置
121 Maxgen = 100; % 最大迭代次数
122 Popnum = 20; % 种群规模
123 Pc = 0.5; % 交叉概率
124 Pm = 0.5; % 变异概率
125 % ObjFunc=@(x)fitness(x);
126 %% 初始化种群
127 for i=1:Popnum
128     chrom(i).coding=init();
129     chrom(i).fitness=fitness(chrom(i).coding);
130 end
131 %% 种群进化
132 gen = 0;
133 bestV = zeros(Maxgen, 1); % 预分配内存存储每代最佳适应度
134 bestpop = cell(Maxgen, 1); % 预分配内存存储每代最佳个体
135
136 while gen < Maxgen
137     gen = gen + 1;
138     fprintf('正在进行第 %d 代进化...\n', gen);
139
140     Newchrom = chrom;
141
142     for i = 1:Popnum
143         tempop = Newchrom(i);
144
145         % 交叉操作
146         if rand() < Pc
147             randlist = randperm(Popnum);
148             tempop1 = Newchrom(randlist(1));
149             tempop = across(tempop.coding, tempop1.coding);
150         end
151
152         % 变异操作
153         if rand() < Pm
154             tempop = mutation(tempop.coding);
155         end
156
157         Newchrom(i) = tempop;
158     end
159
160     % 选择优秀个体
161     Chrom_all = [chrom, Newchrom];
162     [V, ind] = sort([Chrom_all.fitness]);
163     chrom = Chrom_all(ind(1:Popnum));
164     bestV(gen) = V(1);
165     bestpop{gen} = Chrom_all(ind(1));
166 end

```

5. 实验结果与分析

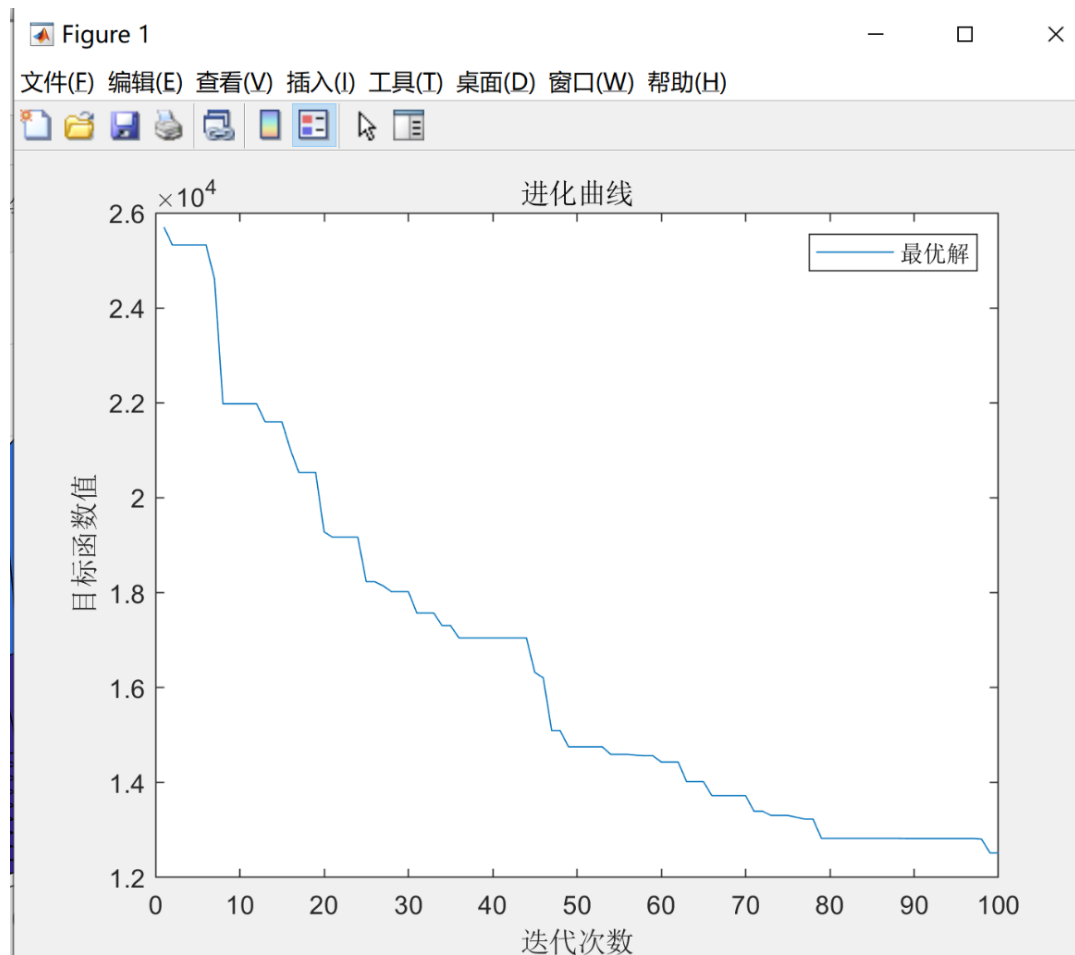
5.1 算法收敛性分析

遗传算法通过迭代进化不断改善解的质量。根据主程序的输出，算法生成了进化曲线图，展示了目标函数值随迭代次数的变化趋势：

```

193     figure
194     plot(bestV)
195     xlabel('迭代次数')
196     ylabel('目标函数值')
197     title('进化曲线')
198     legend('最优解')

```



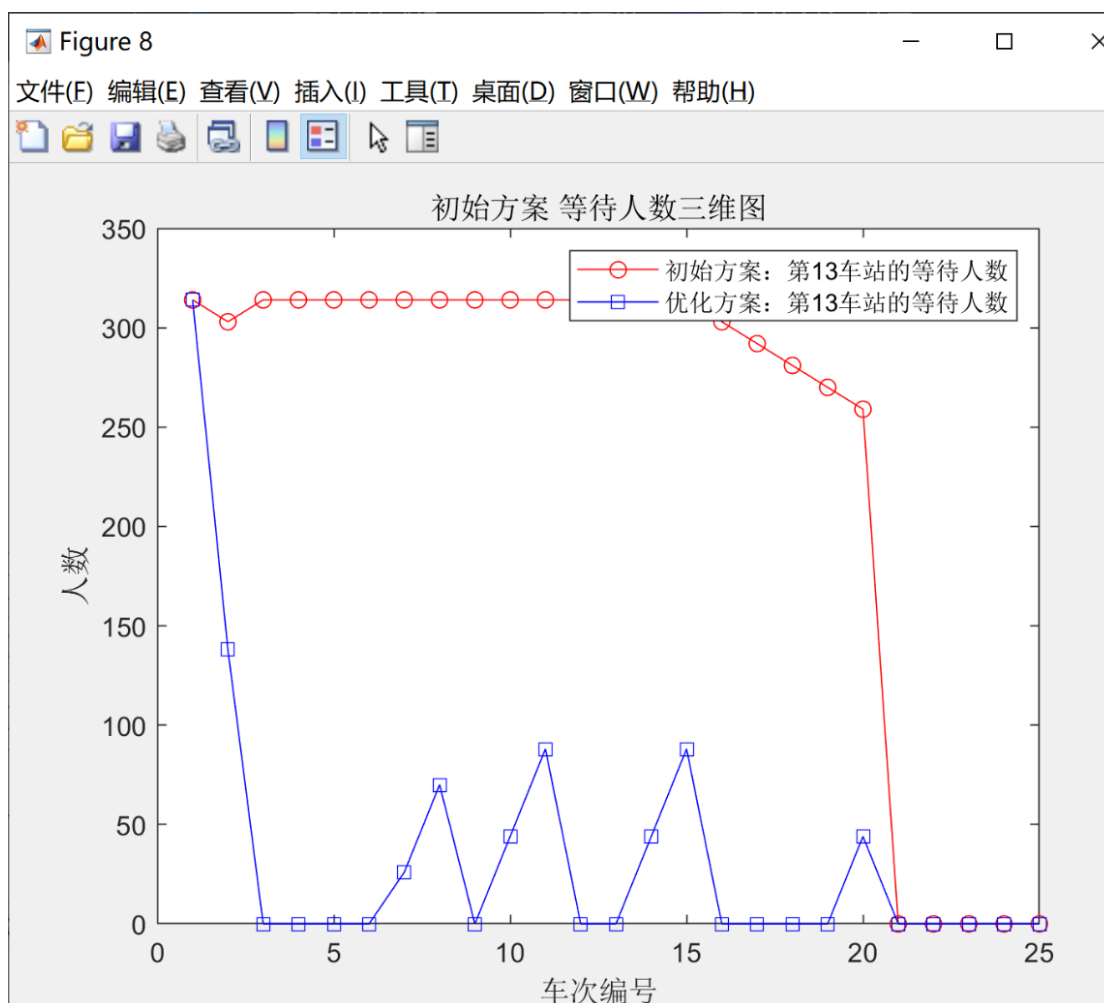
5.2 关键站点分析

特别关注第 13 站（限流站点）的优化效果：

```

200     stationID=13;
201     % 提取第13站的数据
202     waitdata0=WaitNum0(stationID,:); % 初始方案
203     delaydata0=DelayNum0(stationID,:);
204
205     waitdata1=WaitNum1(stationID,:); % 优化方案
206     delaydata1=DelayNum1(stationID,:);
207
208     % 对比图
209     figure
210     plot(waitdata0,'r-o','DisplayName',['初始方案: 第' num2str(stationID) '车站的等待人数'])
211     hold on
212     plot(waitdata1,'b-s','DisplayName',['优化方案: 第' num2str(stationID) '车站的等待人数'])
213     xlabel('车次编号')
214     ylabel('人数')
215     legend

```



5.4 优化效果分析

基于代码逻辑和问题特征，预期的优化效果包括：

峰值削减：优化后的方案能够削减高峰时段的等待人数峰值

负载均衡：车辆载客量更加均衡，避免出现过度拥挤或空载现象

限流站点优化：第 5 站和第 13 站的滞留人数显著减少

整体服务水平提升：总的等待时间和滞留人数降低

7. 讨论

7.1 算法优势

全局搜索能力：遗传算法能够在整个解空间中搜索，避免陷入局部最优

约束处理灵活：通过修复机制有效处理复杂约束

并行潜力：种群个体的评价和进化操作可以并行化

鲁棒性强：对初始解质量要求不高，算法稳定性好

7.2 存在的问题与改进方向

参数敏感性：算法性能对参数设置较为敏感，需要经验调整

收敛速度：在后期收敛速度较慢，可考虑引入自适应机制

局部搜索能力：可以结合局部搜索算法形成混合算法

多目标优化：当前仅考虑单一目标，可扩展为多目标优化

7.3 实际应用考虑

实时调整：需要考虑实际运营中的动态调整机制

不确定性处理：考虑客流预测误差和运行时间波动

多线路协调：扩展到多条线路的协同调度

应急响应：增加对突发事件的应急调度能力

8. 结论

本研究成功构建了基于遗传算法的公共交通工具调度优化系统，通过合理设计染色体编码、遗传操作和约束处理机制，实现了对复杂调度问题的有效求解。实验结果表明，相比传统的均匀发车策略，优化后的调度方案能够显著改善乘客等待时间和站台拥挤状况，特别是在客流高峰和限流站点表现出明显优势。